- امکان دارد پردازه‌ای منبع یکسانی را برای بار دوم (بدون این که، اول آن منبع را رها کند) درخواست نماید.

مشکلاتی مشابه این مسائل، در رابطه با استفاده از سمافورها نیز وجود داشتند و این مشکلات در ذات خود مشابه آنهایی است که در ابتدا ما را تشویق کردند تا بناهای مانیتور را توسعه بدهیم. پیشتر دغدغه‌ی استفاده‌ی درست سمافورها را داشتیم. اکنون، بایستی نگران استفاده‌ی درست عملیات سطح بالای تعریفی برنامه‌نویس باشیم که دیگر کامپایلر هم نمی‌تواند کمکی بکند.

یک راه‌حل ممکن مسأله‌ی فعلی این است که عملیات دسترسی منبع را در داخل مانیتور ResourceAllocator قرار بدهیم. با این وجود، استفاده از این راه‌حل بدان معنی است که زمانبندی به جای آنکه از آنچه کد کرده‌ایم صورت بگیرد، مطابق الگوریتم زمانبندی تعبیه‌شده در داخل مانیتور انجام خواهد گرفت.

برای این که مطمئن شویم، پردازه‌ها دنباله‌های مناسبی را طی می‌کنند، بایستی تمامی برنامه‌هایی را که از مانیتور ResourceAllocator و منبع تحت مدیریت آن استفاده می‌کنند، بازرسی کنیم. برای نهادینه‌کردن صحت این سیستم لازم است دو شرط را بررسی کنیم. اولاً، پردازه‌های کاربر بایستی همواره فراخوانی‌های مانیتور خود را با دنباله‌ی درستی انجام بدهند. ثانیاً، باید مطمئن شویم که یک پردازه‌ی غیرهمکار به راحتی نمی‌تواند با نادیده گرفتن دروازه‌ی انحصار متقابل مانیتور و بدون استفاده از پروتکل‌های دسترسی، یک منبع مشترک را مستقیماً دستیابی کند. تنها با برقراری این دو شرط است که می‌توان مطمئن بود هیچ‌گونه خطای زمانی رخ نخواهد داد و الگوریتم زمانبندی با شکست مواجه نخواهد شد.

اگرچه این بازرسی در یک سیستم کوچک و ایستا امکان‌پذیر است، برای یک سیستم بزرگ یا یک سیستم پویا نمی‌تواند روش مقبولی باشد. مسأله‌ی کنترل دسترسی فقط می‌تواند با استفاده از راهکارهای اضافه‌تری حل شود که در فصل ۱۷ توصیف می‌شوند.

۶-۸ عاقبت به‌خیری

یک پیامد استفاده از ابزار همگامی برای هماهنگی دسترسی به بخش بحرانی، این احتمال است که پردازه‌ای که تلاش دارد به بخش بحرانیش وارد شود به‌صورت نامتناهی منتظر بماند. از بخش ۶-۲ به‌خاطر بیاورید که ما سه معیار مطرح کردیم که راه‌حل‌های مسئله‌ی بخش بحرانی بایستی ارضا نمایند. انتظار نامتناهی دو مورد از این‌ها—معیارهای پیشروی و انتظار محدود—را نقض می‌کند.

عاقبت به‌خیری[1] به مجموعه‌ای از ویژگی‌ها اطلاق می‌شود که یک سیستم بایستی ارضا نماید تا تضمین شود که پردازه‌ها در طی چرخه‌ی حیات اجرایشان پیشروی دارند. انتظار نامتناهی یک پردازه تحت شرایطی که بیان شد مثالی از یک «شکست عاقبت به‌خیری[2]» است.

1. Liveness
2. Liveness failure

اَشکال بسیار متفاوتی از شکست عاقبت به‌خیری وجود دارد؛ با این وجود، همگی به‌صورت کلی با کارایی و پاسخ‌گویی ضعیف مشخص هستند. یک مثال بسیار ساده‌ای از شکست عاقبت به‌خیری، یک حلقه‌ی نامتناهی است. یک حلقه‌ی مشغول منتظر، *احتمال* یک شکست عاقبت به‌خیری را مطرح می‌کند، مخصوصاً اگر یک پردازه در یک حلقه با دوره‌ی زمانی طولانی دلخواهی قرار بگیرد. کارهایی که دنبال تدارک انحصار متقابل با استفاده از ابزاری همچون قفل‌های انحصار متقابل و سمافورهاست می‌تواند به این قبیل شکست‌ها در برنامه‌نویسی همروند منجر شوند. ما در این بخش، دو حالتی را که می‌تواند به شکست عاقبت به‌خیری منجرا شود بررسی می‌کنیم.

۶-۸-۱ بن‌بست

پیاده‌سازی سمافور با استفاده از یک صف انتظار می‌تواند شرایطی را به وجود بیاورد که دو یا چندین پردازه به طور نامعین به انتظار رویدادی بمانند که وقوع آن تنها به دست یکی از همین پردازه‌های منتظر است. رویداد مورد سؤال، اجرای عملیات ()signal است. زمانی که چنین شرایطی پیش می‌آید، گفته می‌شود که این پردازه‌ها در **بن‌بست**[1] قرار دارند.

برای روشن‌شدن مطلب، سیستمی را با دو پردازه‌ی P_0 و P_1 در نظر بگیرید که هر یک، دو سمافور S و Q را با مقادیر ۱ دستیابی می‌کنند:

P_0	P_1
wait(S);	wait(Q);
wait(Q);	wait(S);
.	.
.	.
.	.
signal(S);	signal(Q);
signal(Q);	signal(S);

فرض کنید P_0، wait(S) را اجرا می‌کند و سپس، P_1، wait(Q) را اجرا می‌نماید. لحظه‌ای که P_0، wait(Q) را اجرا می‌کند، به ناچار تا اجرای signal(Q) توسط P_1 منتظر می‌ماند. به طور مشابه، زمانی هم که P_1، wait(S) را اجرا کرد، بایستی تا زمان اجرای signal(S) توسط P_0 منتظر شود. چون این دو عملیات ()signal نمی‌توانند اجرا شوند، پردازه‌های P_0 و P_1 به بن‌بست می‌رسند.

می‌گوییم که مجموعه‌ای از پردازه‌ها در حالت بن‌بست قرار دارند، هرگاه *هر یک از پردازه‌های مجموعه، منتظر رویدادی باشند که تنها توسط پردازه‌ای از همان مجموعه می‌تواند به وقوع بپیوندد.* «رویدادهایی» که در اینجا اساساً دغدغه‌ی آنها را داریم، کسب و رهایی منابعی همچون قفل‌های انحصار متقابل و سمافورها هستند. انواع دیگری از رویدادها نیز ممکن است شرایط بن‌بست را به وجود بیاورند، چنان که در فصل ۸ با جزئیات بیشتر نشان می‌دهیم. در آن فصل، راهکارهای مختلف مواجهه با مسئله‌ی بن‌بست را، در کنار اَشکال دیگر شکست‌های عاقبت به‌خیری مطرح می‌کنیم.

1. Deadlock

۶-۸-۲ وارونگی الویت

یکی از چالش‌های زمانبندی موقعی بروز می‌نماید که یک پردازه‌ی با الویت بالاتر نیاز به خواندن یا اصلاح داده‌هایی را از هسته پیدا می‌کند که فعلاً توسط پردازه‌ای با الویت پایین‌تر— یا زنجیره‌ای از پردازه‌های با الویت پایین‌تر—دستیابی می‌شوند. از آنجایی که داده‌های هسته نوعاً با یک قفل محافظت می‌شوند، پردازه‌ی با الویت بالاتر ناگزیر به انتظار پردازه‌ی با الویت پایین‌تر می‌ماند تا کارش با آن منبع تمام شود. این وضعیت در صورتی‌که پردازه‌ی با الویت پایین‌تر به نفع پردازه‌ی دیگر که الویت بالاتری دارد پس‌گرفته شود، وضع از این هم پیچیده‌تر می‌شود.

به عنوان یک مثال، فرض کنید سه پردازه—L، M و H—داریم که الویت آنها از رابطه‌ی $L < M < H$ تبعیت می‌کند. فرض کنید پردازه‌ی H به منبع R نیاز دارد که فعلاً دست L است. پردازه‌ی H قاعدتاً به انتظار پردازه‌ی L خواهد نشست تا کار آن با منبع R تمام شود. با این وجود، اکنون فرض کنید که پردازه‌ی M، با پس‌گیری L، قابلیت اجرا می‌یابد. به صورت غیرمستقیم، پردازه‌ای با الویت پایین‌تر—پردازه‌ی M—توانسته است، انتظار پردازه‌ی H را برای رهایی منبع R که دست L است طولانی‌تر کند.

این مسأله‌ی عاقبت به‌خیری به **وارونگی الویت**[1] معروف است و تنها در سیستم‌هایی با بیش از دو الویت پیش می‌آید. وارونگی الویت نوعاً با پیاده‌سازی **پروتکل ارث‌بری الویت** قابل اجتناب است. مطابق این پروتکل، تمامی پردازه‌ها که به منابعی دسترسی دارند و مورد نیاز یک پردازه‌ی با الویت بالا می‌باشد، الویت بالاتر را به ارث می‌برند تا این که کارشان با منابع مورد نظر تمام شود. زمانی که کار آنها تمام شد، الویت‌های آنها به مقادیر اولیه بر می‌گردد. در مثال بالا، پروتکل ارث‌بری الویت به پردازه‌ی L اجازه می‌دهد که الویت پردازه‌ی H را موقتاً به ارث ببرد، تا با این کار از پس‌گیری اجرایش توسط پردازه‌ی M جلوگیری کند. زمانی که پردازه‌ی L کارش با منبع R به اتمام رسید، الویت موروثی خود از H را پس داده و با الویت اولیه پیشروی خواهد داشت. چون هم‌اکنون منبع R در دسترس می‌باشد، در ادامه، پردازه‌ی H—و نه M—اجرا خواهد شد.

وارونگی الویت و مسیریاب مریخ

وارونگی الویت می‌تواند بیش از یک نارسایی زمانبندی به حساب بیاید. در سیستم‌های با محدودیت‌های زمانی سفت و سخت، نظیر سیستم‌های بلادرنگ، وارونگی الویت می‌تواند انجام کار یک پردازه را بیش از حد انتظار به تأخیر بیندازد. زمانی که این وضعیت پیش می‌آید، سایر شکست‌ها می‌تواند به صورت آبشاری منجر به از کار افتادن سیستم شود.

مسیریاب مریخ[2]، کاوشگر ناسا را در نظر بگیرید که یک روبات مسافر موقت را در سال ۱۹۹۷ در سطح مریخ فرود آورد تا آزمایش‌هایی را انجام بدهد. مدتی کوتاه بعد از شروع عملیات مسافر موقت نگذشت که کامپیوتر آن شروع به resetهای مکرر نمود. با هر بار reset که صورت می‌گرفت تمامی سخت‌افزار و

1. Priority inversion
2. Mars Pathfinder

نرم‌افزار، از جمله ارتباطات دومرتبه مقدار اولیه می‌گرفتند. اگر این مسأله حل نمی‌شد، مسافر موقت در مأموریتش با شکست مواجه می‌شد.

مسأله از این واقعیت ناشی شده بود که تکمیل انجام کار یک وظیفه‌ی با الویت بالا، به نام «bc_dist» بیش از حد انتظار طول می‌کشید. این وظیفه مجبور به انتظار منبع مشترکی می‌شد که در اختیار یک وظیفه‌ی با الویت پایین به نام «ASI/MET» بود که آن هم توسط چندین وظیفه با الویت متوسط پس‌گرفته می‌شد. وظیفه‌ی «bc_dist» به انتظار منبع مشترک متوقف می‌ماند تا این که، وظیفه‌ی «bc_sched» سرانجام متوجه مسأله می‌شد و سیستم را reset می‌کرد. مسافر موقت از یک مورد خاص وارونگی الویت رنج می‌برد.

سیستم‌عامل مسافر موقت، سیستم‌عامل بلادرنگ VxWorks بود که برای فعال‌سازی ارث‌بری الویت تمامی سمافورها یک متغیر سراسری داشت. بعد از آزمایش، این متغیر به مسافر موقت (بر روی مریخ!) مقداردهی شد و به این ترتیب، مسأله حل شد.

توصیف کاملی از مسأله، تشخیص و راه‌حل آن که با هدایت یک تیم نرم‌افزاری نوشته شده بود، در آدرس http://research.microsoft.com/en-us/um/people/mbj/mars_pathfinder/authoritative_account.html در دسترس می‌باشد.

۶-۹ ارزیابی

ما چندین ابزار همگامی متفاوت را مطرح کرده‌ایم که می‌توانند برای حل مسئله‌ی بخش بحرانی استفاده شوند. این ابزار با پیاده‌سازی و استفاده‌ی صحیح می‌توانند به‌صورت مؤثر برای تضمین انحصار متقابل، در کنار لحاظ ملاحظات عاقبت به‌خیری استفاده شوند. با ازدیاد برنامه‌های همروندی که از قدرت سیستم‌های کامپیوتری چندهسته‌ای مدرن بهره می‌برند، توجه فزاینده‌ای به کارایی ابزار همگامی معطوف می‌شود. با این وجود، شناسایی زمان استفاده از این ابزار می‌تواند یک چالش خطیری باشد. ما در این بخش، برخی راهبردهای ساده‌ی تعیین زمان استفاده از ابزار همگامی را ارائه می‌دهیم.

راه‌حل‌های سخت‌افزاری مطرح شده در بخش ۶-۴ بسیار سطح پایین محسوب شده و نوعاً به‌عنوان مبانی ساخت ابزار همگامی دیگر، همچون قفل‌های انحصار متقابل استفاده می‌شوند. با این وجود، تمرکز اخیری بر استفاده از دستورالعمل CAS برای ساخت الگوریتم‌های **عاری از قفل**[1] که حفاظت شرایط مسابقه را بدون الزام سربار قفل‌گذاری فراهم می‌کنند وجود دارد. هرچند که این راه‌حل‌های عاری از قفل به‌واسطه‌ی سربار کم و توانمندی توسعه‌پذیری در حال فراگیرشدن هستند، توسعه و آزمون خود الگوریتم‌ها غالباً دشوار است. (در تمرین‌های انتهای همین فصل از شما خواسته‌ایم تا صحت پشته‌ی عاری از قفل را ارزیابی کنید.)

رویکردهای مبتنی بر CAS، رویکرد *خوش‌بینانه* محسوب می‌شوند—شما به‌صورت خوش‌بینانه در ابتدا یک متغیر را به‌روز کرده و سپس از تشخیص برخورد برای مشاهده‌ی این که آیا نخ دیگری متغیر را به‌صورت همروند به‌روز می‌کند یا نه، استفاده می‌کنید. اگر این‌گونه باشد، به‌صورت مکرر عملیات را تجدید می‌کنید تا این که بدون برخورد به‌صورت موفقیت‌آمیز به‌روز شود. در مقابل، قفل‌گذاری انحصار متقابل، یک راهبرد *بدبینانه* محسوب می‌شود؛ شما

1. Lock-free

فرض می‌کنید که نخ دیگری به‌صورت همروند در حال به‌روزرسانی متغیر است، لذا شما به‌صورت بدبینانه، قبل از این که هرگونه به‌روزرسانی انجام بدهید قفل را کسب می‌کنید.

خط‌مشی‌های زیر قوانین کلی مربوط به تفاوت‌های کارایی را میان همگامی مبتنی بر CAS و همگامی سنتی (همچون قفل‌های انحصار متقابل و سمافورها)، تحت بارهای رقابتی متنوع مشخص می‌کنند:

- **غیررقابتی:** هرچند که هر دو گزینه در حالت کلی سریع هستند، حفاظت CAS تا حدودی سریع‌تر از همگامی سنتی می‌باشد.
- **رقابت متوسط:** حفاظت CAS سریع‌تر—احتمالاً بسیار سریع‌تر—از همگامی سنتی خواهد بود.
- **رقابت بالا:** تحت بارهای رقابتی بسیار بالا، همگامی سنتی نهایتاً سریع‌تر از همگامی مبتنی بر CAS خواهد بود.

علی‌الخصوص رقابت متوسط برای بررسی جالب است. در این سناریو، عملیات CAS در بیشتر اوقات موفق می‌شود و زمانی که با شکست مواجه می‌شود تنها به‌دفعات معدودی در حلقه‌ی نشان داده شده در شکل ۶-۸ دور خواهد زد قبل از این که موفق باشد. در مقایسه با قفل‌گذاری انحصار متقابل، هرگونه تلاش برای کسب یک قفل رقابتی به یک مسیر کد بغرنج‌تری—و وقت‌گیر—منجر خواهد شد که یک نخ را معلق کرده و آن را در یک صف انتظار می‌گذارد که مستلزم یک برگردان متن به نخ دیگری است.

انتخاب یک راهکاری که شرایط مسابقه را مرتفع می‌سازد می‌تواند به‌صورت قابل توجهی کارایی سیستم را تحت تأثیر قرار دهد. برای مثال، مقادیر صحیح یکپارچه خیلی سبک‌وزن‌تر از قفل‌های سنتی هستند، و عموماً برای به‌روزرسانی‌های تکی متغیرهای مشترک، همچون شمارنده‌ها مناسب‌تر از قفل‌های انحصار متقابل یا سمافورها می‌باشند. ما این موضوع را در طراحی سیستم‌عامل‌ها دیده‌ایم جایی که قفل‌های چرخشی، موقعی که قفل‌های برای فاصله‌های کوتاهی نگه داشته می‌شوند در سیستم‌های چندپردازنده استفاده می‌شوند. در حالت کلی، قفل‌های انحصار متقابل ساده‌تر بوده و سربار کمتری به‌نسبت سمافورها دارند و در حفاظت دسترسی به یک بخش بحرانی به سمافورهای دودویی ترجیح داده می‌شوند. با این وجود، در برخی استفاده‌ها—همچون کنترل دسترسی به یک تعداد منبع متناهی—یک سمافور شمارشی عموماً مناسب‌تر از یک قفل انحصار متقابل است. با تشابهی، در برخی نمونه‌ها، یک قفل خواننده-نویسنده ممکن است به قفل انحصار متقابل ترجیح داده شود، از این نظر که درجه‌ی بالاتری از همروندی را ممکن می‌سازد (یعنی، خوانندگان متعدد).

جذابیت ابزار سطح بالاتری همچون مانیتورها و متغیرهای شرط به سادگی و سهولت استفاده از آن‌ها بر می‌گردد. با این وجود، این قبیل ابزار ممکن است سربار قابل توجهی داشته و بسته به پیاده‌سازی آن‌ها، ممکن است با احتمال کمتری در شرایط رقابتی بالا توسعه‌پذیر باشند.

خوشبختانه، تحقیقات در جریان زیادی به سمت توسعه‌ی ابزار مؤثر توسعه‌پذیر وجود دارد که نیازهای برنامه‌نویسی همروند را مد نظر قرار می‌دهند. برخی از این نمونه‌ها عبارتند از:

- طراحی کامپایلرهایی که کد مؤثرتری تولید می‌کنند.

- توسعه‌ی زبان‌هایی که از برنامه‌نویسی همروند پشتیبانی می‌کنند.
- بهبود کارایی کتابخانه‌ها و APIهای موجود.

ما در فصل بعدی، چگونگی پیاده‌سازی ابزار همگامی ارائه شده در این فصل توسط سیستم‌عامل‌ها و APIهای مختلف در دسترس توسعه‌دهندگان را بررسی می‌کنیم.

۶-۱۰ خلاصه

- یک شرایط مسابقه زمانی پیش می‌آید که پردازه‌ها به داده‌های مشترکی دسترسی داشته و نتیجه‌ی نهایی به ترتیب خاصی بستگی دارد که در آن، دسترسی‌های همروند اتفاق می‌افتند. شرایط مسابقه می‌تواند به خرابی مقادیر داده‌های مشترک بینجامد.
- بخش بحرانی، بخشی از کد است که داده‌های مشترک ممکن است دستکاری شده و یک شرایط مسابقه‌ی احتمالی ممکن است پیش بیاید. مسئله‌ی بخش بحرانی، طراحی پروتکلی است که با آن پردازه‌ها بتوانند فعالیت خودشان را در کار بر روی داده‌های مشترک همگام سازند.
- یک راه‌حل مسئله‌ی بخش بحرانی بایستی این سه نیازمندی را ارضا نماید: (۱) انحصار متقابل، (۲) پیشروی، و (۳) انتظار محدود. انحصار متقابل تضمین می‌کند که تنها یک پردازه در هر لحظه در بخش بحرانیش فعال خواهد بود. پیشروی تضمین می‌کند که برنامه‌ها با همکاری، پردازه‌ای را که در نوبت بعدی وارد بخش بحرانیش خواهد شد تعیین می‌کنند. انتظار محدود، زمان لازم مورد انتظار یک برنامه قبل از ورود به بخش بحرانیش را محدود می‌سازد.
- راه‌حل‌های نرم‌افزاری مسئله‌ی بخش بحرانی، همچون راه‌حل پترسن در معماری کامپیوترهای مدرن خوب کار نمی‌کنند.
- پشتیبانی سخت‌افزاری مسئله‌ی بخش بحرانی مشمول سدهای حافظه؛ دستورالعمل‌های سخت‌افزاری، همچون compare-and-swap؛ و متغیرهای یکپارچه می‌باشد.
- یک قفل انحصار متقابل، انحصار متقابل را با الزام کسب یک قفل توسط یک پردازه قبل از ورود به یک بخش بحرانی و رهایی قفل در خروج از بخش بحرانی فراهم می‌کند.
- سمافورها، مثل قفل‌های انحصار متقابل، می‌توانند برای تدارک انحصار متقابل استفاده شوند. با این وجود، در حالی که یک قفل انحصار متقابل دارای یک مقدار دودویی است که نشان می‌دهد آیا قفل در دسترس می‌باشد یا خیر، سمافور یک مقدار عددی صحیح دارد و از این رو می‌تواند برای حل مسئله‌های همگامی گوناگون استفاده شود.
- مانیتور یک نوع داده‌ی انتزاعی است که شکل سطح بالایی از همگامی پردازه را فراهم می‌کند. مانیتور از متغیرهای شرط استفاده می‌کند که به‌پردازه‌ها اجازه می‌دهند تا برای true شدن شرط‌های خاصی منتظر مانده و موقعی که شرط‌ها true می‌شوند به یکدیگر علامت بدهند.
- راه‌حل‌های مسئله‌ی بخش بحرانی ممکن است گرفتار مسئله‌های عاقبت به‌خیری، از جمله بن‌بست شوند.